OBIETTIVI

- Spiegare tecniche per prevenire l'introduzione di errori nelle vostre soluzioni.
- Discutere come mitigare gli effetti degli errori introdotti.
- Introdurre un approccio sistematico al testing tramite strategie top-down e bottom-up.
- Mostrare come testare le singole parti di una soluzione in isolamento.
- Discutere diverse strategie di debugging.
- Spiegare come gestire i bug trovati nella vostra soluzione.

CONFRONTO CON I BUG

Ricordo bene quando... mi sono reso conto con piena forza che una buona parte del resto della mia vita sarebbe stata spesa a trovare errori nei miei stessi programmi. (Wilkes, 1985)

Dovreste abituarvi all'idea che le soluzioni che create non funzioneranno perfettamente. Conterranno bug da qualche parte, nonostante i vostri migliori sforzi. Anche i programmatori e gli scienziati informatici più talentuosi del mondo producono software che contiene bug, anche se alcuni sono riluttanti ad ammetterlo. Un bug può essere semplice come un errore di battitura in una formula o disastroso come un completo fraintendimento della definizione originale del problema.

Bug: un difetto in una soluzione che può causare comportamenti errati.

I bug possono essere scoperti ed eliminati in qualsiasi fase del processo di sviluppo, ma più presto li trovate e li correggete, meglio è. Troverete molto più facile cambiare qualcosa quando esiste solo come idea piuttosto che dopo che è stata implementata. Pensate allo sforzo che uno scrittore deve mettere per correggere un buco nella trama quando viene trovato durante la fase di pianificazione rispetto a nella bozza finale. Oppure pensate al costo di richiamare e correggere un errore in uno smartphone dopo che ne sono stati venduti milioni. Pertanto, progettare per eliminare i bug è una buona pratica. Questo capitolo esaminerà tali pratiche.

Errore: qualunque comportamento osservato durante l'esecuzione di una soluzione che non corrisponde al comportamento atteso.

Alcuni bug diventeranno evidenti solo dopo che la vostra soluzione (o parti di essa) è stata messa in atto. In questi casi, sono necessari approcci differenti, ovvero test e debug. Anche questi verranno discussi in questo capitolo.

ELIMINARE I BUG NELLA PROGETTAZIONE

Questa sezione esamina i vari tipi di bug che possono causare comportamenti errati.

Errori di battitura

Nelle prime fasi di pianificazione, la vostra soluzione esiste in varie forme di progettazione: documenti, appunti, diagrammi, schemi di algoritmi e così via. Il modo più semplice per prevenire errori in questa fase è cercare errori elementari, sia nella definizione del problema che nella vostra soluzione.

Gli errori di battitura includono:

- errori ortografici;

- maiuscole sbagliate;
- parole mancanti;
- parole sbagliate (ad esempio, "fase" invece di "frase");
- elenchi numerati con numerazione errata, e così via.

Tali errori possono spesso essere banali, ma ognuno di essi potrebbe causare problemi.

Grammatica scadente e ambiguità

Come gli errori di battitura, alcuni tipi di errore grammaticale sono abbastanza facili da individuare e correggere. È facile trovare l'errore in questa frase, vero? Tuttavia, alcuni errori grammaticali sono piuttosto sottili. Per esempio:

Dopo l'inserimento di una moneta, l'utente potrà spingere il tornello.

Chi o cosa esattamente sta inserendo una moneta qui? Affermazioni come queste necessitano di essere riformulate in modo che possano essere interpretate in un solo modo. Non dovreste quindi sottovalutare l'utilità di buone competenze redazionali.

Tuttavia, le ambiguità possono apparire anche in frasi che sono grammaticalmente corrette. Per esempio:

Dopo che il processo di accesso fallisce più volte di seguito, l'incidente dovrebbe essere segnalato tramite email all'amministratore della sicurezza.

L'intento dietro questa affermazione è corretto, ma cosa significa esattamente "un certo numero di volte"? Due, cinque, cento? Potrebbe essere permesso decidere il numero esatto in una fase successiva, ma in generale dovreste definire tali dettagli il più presto possibile.

Un altro tipo di ambiguità che può potenzialmente causare problemi è quando lo stesso concetto (apparentemente) viene riferito usando termini diversi. Questo può sembrare abbastanza innocuo, ma le differenze implicite potrebbero suggerire che ci sono diversi concetti da distinguere o che può essere creata un'astrazione. Ad esempio, se la vostra soluzione fa vari riferimenti a uno "smartphone", "tablet" e "dispositivo portatile" tutti nello stesso contesto, dovreste chiedervi se questi termini possano veramente essere usati in modo intercambiabile o se dovreste trattarli diversamente.

Mantenete un glossario dei termini chiave usati nella vostra soluzione e attenetevi ad esso.

Incoerenze

Eliminare le ambiguità aiuta a contrastare un'altra fonte di bug, ovvero le incoerenze. Un'incoerenza si verifica quando parti diverse di una soluzione fanno affermazioni che non hanno senso se considerate insieme. Questi bug sono più difficili da localizzare perché ogni parte dell'incoerenza potrebbe apparire in contesti molto diversi.

Considerate l'esempio del processo di login (sopra) riscritto come segue: Dopo che il processo di login fallisce cinque volte di fila, l'incidente dovrebbe essere segnalato tramite email all'amministratore della sicurezza. Immaginate anche che una parte diversa della soluzione contenga la seguente affermazione: Dopo che il processo di login fallisce tre volte di fila, la schermata di login dovrebbe automaticamente bloccarsi e rifiutare ulteriori tentativi di accesso.

Qui abbiamo un'incoerenza. Se un tentativo di login fallisce tre volte consecutivamente, l'utente non è in grado di effettuare ulteriori tentativi di login. Di conseguenza, non potrà mai

effettivamente eseguire cinque tentativi di login di fila, rendendo così la prima affermazione superflua.

Errori logici e matematici

Bug semplici possono essere trovati nelle espressioni logiche. Ad esempio, il seguente algoritmo dovrebbe stampare il valore massimo tra due numeri, x e y. Se x > y, allora stampa y Altrimenti se y > x, allora stampa x

Questo contiene in realtà due bug. Uno dovrebbe essere ovvio: l'algoritmo stampa il minimo invece del massimo. Il secondo è leggermente più sottile: cosa succede se entrambi i numeri sono uguali? Ci si aspetterebbe che venga stampato un valore, ma in realtà, non viene stampato nulla. Questo esempio dimostra che gli errori possono derivare da espressioni che sono valide (cioè possono essere valutate senza problemi) ma il cui significato semantico è errato.

Allo stesso modo, errori matematici possono derivare anche da espressioni apparentemente valide. Un computer di bordo che valuta l'efficienza dell'auto basata sul contatore del viaggio potrebbe usare una formula come questa: e = m / g dove l'efficienza (e) è il miglia percorse (m) diviso i galloni di carburante consumati (g). Questo funzionerà per lo più bene, tranne quando il contatore del viaggio viene azzerato, i galloni consumati diventano zero e dividere per zero è matematicamente non valido. La soluzione dovrebbe prendere in considerazione questo caso speciale.

Essere esaustivi

Il capitolo 2 ha evidenziato alcune delle limitazioni dell'interazione uomo-computer. Ecco un promemoria: i computer non hanno buon senso; non cercheranno di colmare eventuali lacune nelle tue istruzioni e non possono condurre un dialogo con te per definire esattamente ciò che intendevi dire.

Il percorso che un computer segue attraverso un algoritmo varia in momenti diversi. Dipende tutto dai parametri passati, così come da altri fattori variabili nell'ambiente. Pertanto, gli algoritmi dovrebbero essere esaustivi e contenere istruzioni per tutti gli esiti concepibili. Se non prendi in considerazione alcuni di essi, il comportamento del computer diventa imprevedibile: potrebbe intraprendere un'azione indesiderata o potrebbe non fare nulla quando in realtà dovrebbe fare qualcosa.

Questi tipi di errori sono facilmente commessi e sono comuni tra i principianti (Pea et al., 1987; Pane et al., 2001). Anche i professionisti commettono tali errori, più di quanto vorrebbero ammettere. Scrivere un algoritmo robusto che tenga conto di tutti i percorsi possibili (sia desiderati che indesiderati) richiede esperienza, ma puoi imparare alcuni principi generali così come come evitare alcuni errori comuni:

- Per ribadire un consiglio del Capitolo 2: pensa criticamente. Una volta che hai scritto una sequenza di istruzioni che ti aspetti vengano seguite, inizia a domandarti cosa potrebbe andare storto. Esamina ogni passaggio e chiediti come potrebbe essere possibile che le cose si svolgano diversamente da come desideri. Se necessario, aggiungi istruzioni che intervengano in caso di risultati indesiderati.
- Guarda soprattutto alle condizionali:
    - Se hai una sequenza di istruzioni nella forma "se X è il caso, allora fai Y", chiediti cosa succede se X non è il caso? A volte è necessaria una

contromisura. Pertanto, sii sempre consapevole di Z – come in, "se X è il caso, allora fai Y, altrimenti fai Z".

- Le condizioni composte possono essere problematiche (ad esempio, se A e non B o C...). Pensa a tutte le possibili combinazioni e sii consapevole che il numero di combinazioni aumenta esponenzialmente man mano che una condizionale aumenta di dimensioni.

Un vecchio e fidato consiglio dice che i 'bug si annidano negli angoli e si radunano ai confini' (Beizer, 1990). Prendi questo esempio: 'se il punteggio è < 50, allora la performance è OK; se il punteggio è > 50, allora la performance è buona'. Cosa manca? Non si tiene conto del caso in cui il punteggio sia esattamente 50. Il consiglio ti suggerisce di prestare particolare attenzione quando testi intervalli e confini.

Guarda anche particolarmente ai cicli:

- Se un ciclo può essere interrotto anticipatamente, inserisci un'istruzione esplicita per farlo nel punto corretto.
- Se il resto di un'iterazione di un ciclo può essere saltato e il computer può passare alla prossima iterazione, inserisci nuovamente un'istruzione esplicita per farlo nel punto corretto.
- Assicurati assolutamente che il ciclo non possa continuare all'infinito.

Consigli generali

Come autore della soluzione, non sei un osservatore imparziale. Ti sarà difficile vedere eventuali problemi che esistono, in parte perché sei troppo immerso nei dettagli di come dovrebbe funzionare e in parte perché non vuoi pensare alla tua amata creazione come difettosa. Potresti anche, senza rendertene conto, aver fatto delle ipotesi implicite che si rivelano ingiustificate. È quindi una buona idea trovare un amico o un collega che esamini la tua soluzione proposta e esegua questi controlli in aggiunta a te.

Nel Capitolo 3 è stato raccomandato di affrontare la risoluzione dei problemi in modo iterativo. Ora è un buon momento per ribadire quel consiglio. Quando progetti la soluzione, fallo a piccoli passi. Lavora alla soluzione per fasi, completando ogni iterazione applicando questi controlli.

Fai del tuo meglio per mantenere le diverse parti della tua soluzione autocontenute minimizzando il numero di relazioni tra di esse. Questo dovrebbe localizzare gli effetti di eventuali errori e aiutare a prevenire effetti a catena.

Infine, se la definizione del problema ti è stata fornita da qualcun altro, chiedi a quella persona di verificare il tuo progetto di soluzione. Gli equivoci tra il progettista della soluzione e il 'cliente' sono una fonte notoria di problemi.

MITIGARE GLI ERRORI

Finora, questo capitolo ha esaminato alcune azioni preventive che puoi intraprendere per impedire che i bug si insinuino nella tua soluzione. La brutta notizia è che la tua soluzione finita conterrà molto probabilmente dei bug, nonostante i tuoi migliori sforzi. La buona notizia è che esistono passi che puoi compiere contro gli errori. Le sezioni seguenti esaminano cosa puoi fare per mitigarne gli effetti.

Difendersi

Un modo per minimizzare gli effetti degli errori è anticipare dove potrebbe verificarsi un problema e poi mettere in atto delle 'barriere protettive'. L'informatica chiama questo programmazione difensiva. L'approccio verifica che certe condizioni siano come previsto prima di eseguire il compito principale della soluzione. Un mancato rispetto di queste condizioni potrebbe indicare qualsiasi cosa, da un avviso a un potenziale fallimento del sistema.

Come per altre pratiche in CT, possiamo vedere questo stesso approccio anche in altre discipline. I sistemi fisici dove la sicurezza è una questione importante sono progettati difensivamente. Cioè, sono sicuri per impostazione predefinita. Lo stato predefinito di un ascensore senza energia è avere i freni attivati. Per far muovere l'ascensore, è necessario fornire energia ai freni per disattivarli. Se l'ascensore perde energia, ritorna al suo stato predefinito e senza potenza con i freni applicati. Quindi, quando qualcosa va storto, le persone all'interno dell'ascensore sono automaticamente tenute al sicuro senza alcun bisogno di intervento. In sostanza, l'ascensore ha bisogno di un 'permesso' esplicito per svolgere le sue normali operazioni.

Puoi far comportare la tua soluzione in modo analogo all'ascensore. Prima che la soluzione compia un passo potenzialmente errato, fai sì che verifichi un insieme di precondizioni prima che il 'normale business' possa procedere. Ad esempio, se un passaggio divide due numeri, sai che fallirà se il divisore è zero. Pertanto, puoi creare una precondizione che controlli questo valore. Se non è zero, allora il normale business può procedere.

Reagire ai problemi

Cosa fare in caso di mancata precondizione dipende dal contesto e, in particolare, dalle conseguenze di un errore. Se la gravità è bassa, l'azione intrapresa potrebbe essere qualcosa di minore come mostrare un messaggio. Se la gravità è alta e rischia di causare danni reali, il sistema dovrebbe entrare in una sorta di modalità di emergenza o addirittura spegnersi.

Ad esempio, il normale business per una macchina distributrice di bevande è leggere il valore del denaro inserito e servire al cliente la loro scelta di bevanda. Ma diverse cose potrebbero andare storte e le risposte programmate variano:

- Il cliente potrebbe inserire denaro illeggibile o straniero. La macchina si riprende semplicemente restituendo il denaro (idealmente con un messaggio utile al cliente).
- Una macchina distributrice particolarmente intelligente potrebbe avere la capacità di rilevare quando un bicchiere è caduto sul vassoio di erogazione. In questo caso, la macchina dovrebbe rifiutarsi di continuare a funzionare fino a quando il bicchiere non è raddrizzato, dopodiché ritorna al normale business.
- Una perdita nell'impianto idrico della macchina distributrice è un problema grave, poiché rischia di far defluire acqua sul pavimento o negli impianti elettrici della macchina. Se viene rilevata una perdita, la cosa prudente è probabilmente spegnere la macchina.

Altre considerazioni generali quando si reagisce agli errori includono:

- se il sistema è interattivo o meno (che determina se può chiedere all'utente cosa fare);
- se il sistema è transazionale o meno. Se lo è, allora è possibile annullare le modifiche effettuate durante un passaggio difettoso.

Verifica dell'input dell'utente

Nei casi in cui la tua soluzione richiede un input da parte dell'utente, un po' di 'paranoia salutare' è molto utile. Gli umani commettono errori tutto il tempo (e gli utenti malevoli lo fanno intenzionalmente), quindi è meglio presumere che il tuo utente inserirà input errati in qualche momento. Pensa a tutti i modi in cui un input potrebbe essere sbagliato:

- Se la tua soluzione si aspetta un numero, verifica che sia realmente un numero prima di fare qualsiasi cosa con esso. Inserire '9O' invece di '90' è un errore abbastanza facile da fare.
- Allo stesso modo, se la tua soluzione richiede un input entro un certo intervallo, controllalo dopo che è stato inserito. Assicurarsi che un numero di conto sia lungo gli otto cifre previste è rapido e facile. Tutto quello che devi fare è contare il numero di cifre nell'input e assicurarti che sia la quantità giusta.
- Formati errati: numeri di telefono, URL dei siti web, indirizzi email, date e orari sono tutti esempi di dati che devono corrispondere a un formato specifico.

Essere proattivi e proteggersi contro comportamenti indesiderati è essenziale. Tuttavia, alcuni bug si insinueranno nonostante i tuoi sforzi.

TESTING

Questa sezione descrive il testing, un metodo per localizzare i bug nascosti in un sistema completamente o parzialmente funzionante.

Obiettivo

L'obiettivo del testing è utilizzare il sistema reale che hai prodotto per verificare se funziona come previsto e per identificare aree specifiche in cui fallisce. Pertanto, il testing deve attendere fino a quando la tua soluzione non ha raggiunto un certo stato di funzionamento. Potrebbe essere completamente funzionante, il che significa che puoi testare l'intero sistema, o potrebbe essere parzialmente funzionante, permettendoti di testare singole parti.

Per eseguire efficacemente il testing, devi effettivamente sapere cosa mira a fare la soluzione. Questo rende essenziale il consiglio del Capitolo 3 – definire chiaramente l'obiettivo della tua soluzione. Guadagni poco verificando che un sistema funzioni come previsto quando hai aspettative mal definite. L'obiettivo della soluzione funge da guida per il testing.

Approccio

È molto importante nel testing essere sistematici. Un testing ad hoc che non segue un approccio particolare è facile da eseguire, ma l'unica cosa che probabilmente imparerai è che c'è qualcosa di sbagliato da qualche parte. Essere sistematici ti aiuterà a localizzare il problema e scoprire esattamente cosa non va e dove.

Puoi scegliere tra due approcci al testing, a seconda di ciò che vuoi ottenere. Il primo è il testing dall'alto verso il basso (top-down testing), dove testi la soluzione nel suo insieme per assicurarti che funzioni. Questo è più efficace per trovare difetti di progettazione e verificare che il sistema funzioni bene insieme. Il secondo approccio è dal basso verso l'alto (bottom-up testing), che richiede di iniziare testando le parti più piccole della soluzione individualmente. Questo ti permette di verificare che esse adempiano correttamente ai loro obblighi all'interno dell'intero sistema e di mostrare che la tua soluzione è costruita su basi solide.

Tabella 5.1 Vantaggi e svantaggi del testing dall'alto verso il basso (top-down) e dal basso verso l'alto (bottom-up)

Vantaggi

- **Top-down**
    - Efficace per trovare difetti di progettazione.
    - Può essere incoraggiante testare un sistema completo, anche se è incompleto o contiene bug.
    - Facile da applicare nelle prime fasi dello sviluppo.
- **Bottom-up**
    - Efficace per localizzare problemi.

Svantaggi

- **Top-down**
    - Difficile da applicare quando il sistema è incompleto.
    - Richiede di usare dei segnaposto per le parti non finite per simulare il loro comportamento.
    - Più difficile localizzare i bug quando si verificano errori.
- **Bottom-up**
    - Richiede di simulare le parti di controllo della soluzione.
    - Non espone difetti di progettazione fino a molto più tardi.

Se possibile, dovresti combinare questi approcci quando effettui il testing, ottenendo così il meglio di entrambi i mondi. Quando inizi a implementare la tua soluzione, puoi iniziare mettendo insieme una struttura scheletrica riempita con segnaposto. Questa dovrebbe essere eseguibile, anche se potrebbe non fare ancora nulla di utile. Successivamente, puoi iniziare a implementare i pezzi individuali, sostituendo ogni segnaposto con una parte funzionante che puoi subito testare in isolamento (questo è il testing bottom-up). Allo stesso tempo, poiché hai una struttura scheletrica funzionante, puoi testare l'intero sistema per assicurarti che funzioni ancora come previsto (testing top-down).

Testare parti individuali

Se hai seguito i consigli nei capitoli 3 e 4, in modo che la tua soluzione esista in pezzi ragionevolmente indipendenti, il testing sarà semplificato. Testando solo piccole parti della soluzione alla volta, faciliti la localizzazione del problema. I test che esercitano solo parti individuali sono chiamati test unitari. Tuttavia, testare parti individuali può essere complicato e merita una spiegazione. La difficoltà deriva dal fatto che le parti individuali della tua soluzione normalmente operano come parte di un insieme più grande e quindi presuppongono che il resto del sistema sia presente e corretto. Questo non è il caso quando vengono testate in isolamento, quindi devi simulare qualsiasi cosa di cui la parte è dipendente.

Ad esempio, alcuni componenti si aspettano di ricevere parametri come parte del loro lavoro normale, quindi per testarli dovrai fornire alcuni dati fittizi che simulino un'operazione

normale. Nel caso ideale, dovresti ideare ogni valore concepibile diverso che un parametro potrebbe avere per testare come la parte risponde in ogni situazione possibile.

per testare le tue ipotesi. Per esempio, in una classe di equivalenza che include numeri divisibili per 3, potresti scegliere un numero rappresentativo come 3, 6, o 9, e verificare che la subroutine risponda correttamente con "Fizz". In modo simile, per i numeri divisibili per 5, potresti scegliere 5 o 10 e assicurarti che la risposta sia "Buzz". Per i numeri divisibili sia per 3 che per 5, come 15 o 30, la risposta attesa sarebbe "FizzBuzz".

Questa metodologia ti permette di condurre un testing efficiente coprendo possibili scenari rappresentativi senza dover testare ogni singolo valore. Ciò diventa particolarmente utile quando hai a che fare con un grande insieme di dati di input, dove testare ogni possibile input diventerebbe impraticabile.

Applicazione della Teoria delle Classi di Equivalenza nel Testing

Quando applichi la teoria delle classi di equivalenza nel testing, segui questi passi:

1. **Identifica i Requisiti**: Determina quali sono i requisiti di input della componente che stai testando. Questo può includere tipi di dati, range di valori, e altre specifiche funzionali.
2. **Definisci le Classi di Equivalenza**: Basato sui requisiti, definisci insiemi di valori di input che si prevede abbiano lo stesso comportamento (risposta). Ogni classe dovrebbe rappresentare una condizione specifica che influisce sul comportamento della componente.
3. **Seleziona Dati di Test Rappresentativi**: Per ogni classe di equivalenza, scegli uno o più valori di test che ritieni rappresentativi di quella classe. Questo riduce significativamente il numero di test da eseguire pur mantenendo una buona copertura delle possibili variazioni nei dati di input.
4. **Esegui i Test**: Testa la componente con i dati di test scelti. Assicurati di includere anche classi di input che testano comportamenti non desiderati o errati per verificare la robustezza della componente.
5. **Valuta i Risultati**: Analizza i risultati dei test per ogni classe di equivalenza. Verifica se la componente si comporta come previsto per ogni gruppo di test. Questo aiuta a identificare problemi specifici che potrebbero non essere evidenti con altri metodi di testing.

Questo approccio non solo rende il processo di testing più gestibile ma anche più mirato, aumentando la probabilità di scoprire errori significativi in fasi più tempestive del ciclo di sviluppo.

Per il gioco FizzBuzz, possiamo dividere tutti gli input in diverse classi di equivalenza, scegliere un valore rappresentativo da ciascuna classe, fornirli alla subroutine e poi verificare che in ciascun caso venga restituita la risposta corretta. Le classi includono:

- **Numeri accettabili normali** (ad esempio, 8): rappresenta tutti i numeri accettabili non divisibili per 3 o 5. La subroutine dovrebbe restituire lo stesso numero che abbiamo dato. Se questo numero funziona, implica che tutti gli altri numeri in questa classe funzionano anche.
- **Fizzes** (ad esempio, 3): rappresenta tutti i numeri divisibili per tre. La subroutine dovrebbe restituire ‘Fizz’.

- **Buzzes** (ad esempio, 10): rappresenta tutti i numeri divisibili per cinque. La subroutine dovrebbe restituire ‘Buzz’.
- **FizzBuzzes** (ad esempio, 30): rappresenta tutti i numeri divisibili sia per tre che per cinque. La subroutine dovrebbe restituire ‘FizzBuzz’.
- **Numeri inaccettabili** (ad esempio, -10): i numeri fuori dall'intervallo dovrebbero essere rifiutati.
- **Non numeri** (ad esempio, VII): solo numeri arabi, per favore. Un computer considera questo come un pezzo di testo e dovrebbe rifiutarlo.

E, perché "i bug si annidano negli angoli e si radunano ai confini" (Beizer, 1990):

- **Limite accettabile** (ad esempio, 1): questo dovrebbe funzionare.
- **Limite inaccettabile** (ad esempio, 0): questo dovrebbe essere rifiutato.

Il testing è un ottimo modo per beneficiare di un altro paio di occhi. Dovresti registrare tutti i tuoi test in un documento (un piano di test). Rendili chiari e facili da capire. Poi, dai quel piano a una terza parte e fai testare la tua soluzione. Un'altra prospettiva ti aiuterà a trovare problemi che potresti non notare da solo.

DEBUGGING

Conosci i miei metodi. Applicali. Sherlock Holmes in "Il segno dei quattro" (Conan Doyle, 1890)

Inevitabilmente, in qualche momento durante il testing o l'uso reale della tua soluzione, si verificherà un errore enigmatico. Non sarà chiaro quale specifico passaggio della soluzione abbia causato il problema, ma non c'è bisogno di entrare in panico. Ancora una volta, puoi essere sistematico nel trovare e correggere, cioè nel fare il debugging, dell'errore. Ci sono metodi che puoi applicare.

Prendi ispirazione da Sherlock Holmes, che considerava la rilevazione una scienza. Il tuo obiettivo è spiegare i fatti (cioè gli errori specifici che si sono verificati) e ricostruire la catena di eventi che ha portato a essi. Devi cercare indizi e usarli per fare deduzioni. Questi ti aiuteranno a costruire un'ipotesi che potrebbe spiegare il problema. Una volta che hai quella, puoi metterla alla prova. Se il tuo test ha successo, avrai trovato gli errori nella tua soluzione e potrai procedere a pensare a una correzione.

Questo è l'approccio generale. In realtà, non esiste un metodo garantito per il debugging, ma sono disponibili numerose strategie diverse (elencate di seguito) per costruire e testare la tua ipotesi. Avrai bisogno di un po' di creatività e immaginazione per aiutarti. Naturalmente, diventerai più bravo nel debugging più lo pratichi.

Sii Implacabile con le Intuizioni Se hai un'intuizione sul problema, provaci. Hai creato il sistema, quindi dovresti avere una buona idea di come funziona e di come potrebbe andare storto. Tuttavia, se la tua intuizione si rivela errata, scartala immediatamente! Non affezionarti troppo alla tua spiegazione preferita.

Rispetta il Rasoio di Occam Potresti avere diverse idee su cosa potrebbe causare il problema. Alcune potrebbero essere semplici, altre complesse. Il Rasoio di Occam è un principio nella risoluzione dei problemi che ci dice che l'ipotesi più semplice - quella che richiede di fare meno assunzioni - è molto probabilmente quella corretta. In altre parole,

resisti alla tentazione di immaginare spiegazioni complicate. Prova prima quelle semplici ed elimina quelle.

Dividi e Conquista Se non sei sicuro di dove risieda il problema, semplificati la vita riducendo il numero di posti che devi cercare. Se, durante la progettazione della tua soluzione, hai suddiviso il tuo problema utilizzando un approccio di divide et impera, ora puoi sfruttare quello stesso approccio per il debugging. Dividere e conquistare ti permette di eliminare intere aree del tuo sistema come sospette in una volta sola. La struttura del tuo sistema - i componenti e le loro relazioni - dovrebbe assomigliare a una struttura ad albero. Man mano che il tuo sistema si esegue, il controllo passa da un componente all'altro. Di conseguenza, se il problema che stai investigando sorge prima che il controllo passi a un certo componente, allora puoi eliminare quel componente come causa del tuo problema. Inoltre, qualsiasi componente che viene invocato solo da quel componente eliminato può essere eliminato anche esso.

Cambia una Cosa alla Volta Un buon debugging è come un buon esperimento. Quando uno scienziato esegue un esperimento, sta cercando la causa di qualcosa. Nel loro caso, variando alcuni fattori e analizzando le conseguenze, stabiliscono un collegamento tra causa ed effetto. Tuttavia, uno scienziato fa attenzione a variare solo un fattore alla volta. Dopotutto, se modificano tre fattori contemporaneamente e poi rilevano con successo un effetto, come possono sapere quale dei tre fattori è stato responsabile? Applica la stessa lezione durante il tuo debugging. Se pensi che una modifica possa risolvere il tuo problema, applicala e poi testala. Se non funziona, annulla la modifica e riporta il sistema allo stato originale prima di provare qualcos'altro.

Logging

In alcuni casi, potresti notare che la tua soluzione fa qualcosa di inaspettato; o fa qualcosa che non intendevi o non fa qualcosa che avrebbe dovuto fare. Pertanto, è necessario indagare quali istruzioni nel tuo sistema vengono eseguite e in quale momento. Ma poiché l'esecuzione avviene all'interno del computer, non sei in grado di "vederla".

Tuttavia, puoi ottenere una sorta di panoramica inserendo un'istruzione di logging aggiuntiva da qualche parte nel tuo algoritmo. Quando questa istruzione viene eseguita, segnala l'evento (ad esempio, stampando un messaggio). L'istruzione di logging agisce come un tornello; ogni volta che il computer "passa attraverso", questo fatto viene segnalato. Ad esempio, potresti aspettarti che il computer passi attraverso un determinato loop cinque volte, quindi aggiungi un'istruzione di log all'interno del loop che fa altro non che stampare la parola "Loop". Se, quando esegui il sistema, il risultato finale è:

Loop

Loop

Loop

Loop

sai che il loop è stato eseguito solo quattro volte.

Puoi fare cose simili se sospetti di certi valori. Potresti notare che la tua soluzione fa qualcosa di indesiderato, facendoti sospettare che un valore errato sia la causa. Non è insolito, ad esempio, che un blocco di istruzioni non si attivi quando ti aspettavi.

if x > 10, then do stuff

Se, quando esegui il programma, le istruzioni rappresentate da "stuff" non vengono eseguite, cosa sospetteresti che causi il problema? Una forte possibilità è che la variabile x sia in realtà minore o uguale a 10 quando questa riga viene eseguita. Potresti confermare ciò aggiungendo un'istruzione di log (immediatamente prima dell'istruzione if) che stampa il valore di x. Una volta stabilito il valore effettivo, puoi poi indagare perché x ha assunto questo particolare valore.

Tracciamento

A volte l'unico modo per diagnosticare un problema è andare riga per riga attraverso ogni istruzione e vedere di persona cosa sta effettivamente facendo il computer.

Puoi farlo da solo a mano, fingendo di essere il computer. Dovrai passare attraverso ogni istruzione, eseguirla e tenere traccia delle modifiche a qualsiasi dato che l'algoritmo mantiene. La Figura 5.2 mostra un esempio di algoritmo da un gioco del Campo Minato dove la scacchiera è divisa in quadrati. Ogni quadrato può essere o meno adiacente a una mina nascosta. Il gioco calcola il numero di mine nelle immediate vicinanze di un quadrato procedendo in senso orario attraverso ogni quadrato adiacente e contando il numero di mine. La Figura 5.2 mostra qualcuno a metà del tracciamento, avendo già attraversato il loop "repeat-until" sei volte. Hanno eseguito ogni linea e tenuto traccia di ogni cambiamento alle due variabili.

Una volta che il tuo sistema è implementato per un computer, puoi anche eseguire il tracciamento utilizzando un debugger. Questo è uno strumento assistito dal computer che esegue un programma sotto il tuo attento controllo. Essenzialmente, ti rivela cosa sta succedendo nella mente del computer mentre esegue la tua soluzione. Puoi usare un debugger per esaminare le istruzioni una per una e stampare lo stato corrente dei vari dati, anche se molti debugger hanno molte altre funzionalità avanzate. L'uso dei debugger è esplorato più dettagliatamente nel Capitolo 12.

Spiega il Problema Ad Alta Voce

Ogni volta che un errore inspiegabile ti lascia perplesso, prova a spiegare il problema a voce alta. Passa attraverso ciò che non va, cosa hai già provato e cosa non hai provato. Potresti essere sorpreso da quanto rapidamente ti viene in mente una nuova spiegazione dopo aver messo il problema in parole.

Questo può essere efficace anche quando lo fai da solo (non dirò a nessuno che parli da solo). Ma per i migliori risultati, spiega il tuo problema a qualcun altro. Non importa se l'altra persona non capisce completamente la tua soluzione. In realtà, è meglio se non la capiscono perché l'altra persona potrebbe inconsapevolmente aiutarti a rispettare il Rasoio di Occam. Il problema potrebbe essere così semplice che è necessaria una domanda "sciocca" per esporre l'errore semplice che hai commesso.

NON PUOI AVERE TUTTO: DECIDERE QUALI ERRORI CORREGGERE

Ogni pezzo di software non banale nel mondo contiene bug. La tua soluzione non sarà diversa. Puoi continuare a testare e fare debugging fino a quando le vacche tornano a casa, ma ci saranno sempre difetti.

È importante riconoscere questo fatto. Ci costringe a smettere di intraprendere una ricerca infinita della perfezione e, invece, a gestire l'elenco dei problemi. Se non tutti i difetti possono essere riparati, allora abbiamo bisogno di un modo per scegliere quali correggere. Gli sviluppatori di software hanno accumulato una vasta esperienza nella gestione degli errori. Questa sezione esamina alcune di quelle esperienze.

La prima domanda da fare riguardo a un errore è: qual è il suo livello di gravità? Questo descrive quanto fortemente l'errore impedisce l'uso corretto del sistema. Ad esempio, un bug che causa il fallimento del sistema ha una gravità alta; quando un logo è colorato leggermente in modo errato, quella è una gravità bassa. I valori di gravità sono tipicamente "Minore", "Moderata", "Maggiore" e "Critica".

La domanda successiva da fare è: qual è la sua priorità? Questa è una domanda leggermente più complessa da rispondere perché devi prendere in considerazione diversi fattori:

- **Frequenza**: più spesso si verifica l'errore, maggiore dovrebbe essere la priorità del bug.
- **Valore della funzionalità**: se il bug riguarda una funzionalità non importante, può avere una bassa priorità. Se una funzionalità chiave è compromessa, il bug dovrebbe avere una priorità alta.
- **Pianificazione**: stima quanto tempo ci vorrà per correggere un errore e chiedi se ne vale la pena. Vale la pena correggere un difetto minore e raro se si stima che richiederà un'intera settimana? D'altra parte, quasi ogni quantità di sforzo impiegata per correggere un fallimento del sistema che si verifica regolarmente è tempo ben speso.

I valori di priorità sono tipicamente "Bassa", "Media" e "Alta" come mostrato nella Figura 5.3.

Questa matrice aiuta a valutare e decidere quali errori affrontare per primo basandosi sulla loro urgenza e impatto, facilitando la gestione delle priorità nello sviluppo del software.

Posizionare i bug su un asse, come nella Figura 5.3, ci aiuta a ordinarli e scegliere quali correggere per primi. È chiaro che i bug che sono ad alta priorità/alta gravità dovrebbero essere classificati come i più importanti, mentre quelli a bassa priorità/bassa gravità dovrebbero essere classificati per ultimi. Per le due categorie intermedie, suggerirei di affrontare prima i problemi ad alta priorità/bassa gravità.

RIASSUNTO

I bug sono errori che possono causare problemi nella tua soluzione. Esistono numerose tecniche sia per prevenire l'introduzione di bug nella tua soluzione sia per affrontarli dopo che si sono insinuati.

Prevenire che i bug entrino nella tua soluzione significa impegnarsi in un attento sforzo di progettazione. Più presto vengono trovati i bug, meno costoso è ripararli.

Mitigare gli errori significa anticipare e controllare le loro potenziali conseguenze. Programmare in modo difensivo è un ottimo modo per prevenire che i bug causino danni.

La ricerca e la rimozione dei bug si effettuano tramite testing e debugging. Il testing ti permette di cercare i bug e sia le strategie top-down che bottom-up offrono modi sistematici di approcciare il testing. Il debugging, utilizzando log o tracciamenti, ti aiuta a rintracciare la causa ultima degli errori.